CANDIDATE ANSWER FORM

Candidate Name: Fabrice RIMLINGER

Role Applied For: Sr Project Mgr Strategy & Risk, EU Supply Chain Risk Management Interview

Date: April 27 / 28 2022

Written Exercise Deadline Date: April 27 2022

---

2. What is the most inventive or innovative thing you have done? Describe something that was your idea, e.g. a process change, a product idea, a new metric, or a novel customer interface. It does not need to be something that is patented. Do not write about anything your current or previous employer would deem confidential information. Provide relevant context for us to understand the invention/innovation. What problem were you seeking to solve, and what was the result? Why was it an important problem to solve? How did it make a difference to the business or organization?

Purpose
This document introduces the Open-source Add-in "Sparklines for Excel" (SfE), an innovative library of 14 in-cell charting formulas for Microsoft Excel.

Context
As Technical Projects Manager, my job required an extensive use of Excel for analysis and reporting of my project portfolio, including KPIs and P&L metrics.
On these dashboards and reports, I tend to use many charts to communicate effectively clear information, for project stakeholders to take sound decisions.
One issue I repeatedly faced while creating ad-hoc reports or updating dashboards is the ridiculous amount of time spent in formatting charts (colors, lines, titles etc), or placing them properly on a spreadsheet. Not mentioning the various intermediary calculations and tweaks that need to be applied in order to create an appropriate readable chart.

Requirements
I needed a tool that could simplify and speed up my reporting process while freeing time for more value-added tasks. I imagined a limited set of simple and efficient charts. The charts had to be easy to implement and modify without constant manual tweaks. The closest analogy is the Lego brick: individually extremely simple, yet easy to assemble with other bricks to build an elaborated object.
Curious about data visualization, one of my sources of inspiration was Edward Tufte's book "*The visual display of quantitative Information*".
Tufte has elaborated a set of rules for efficient analytical charting that I tend to follow:

- Above all else show the data.
- Remove all unnecessary cosmetics (3D, colors etc…)
- Graphics can be shrunk way down in order to densify information
- Prefer small multiple charts than large and complex graphics
- Graphical elegance is often found in simplicity of design

Tufte also introduced the concept of Sparkline, described as "a small intense, simple, word-sized graphic with typographic resolution". The Sparkline would be my Excel data visualization Lego brick.

Strategy
At first, I investigated if there were already Sparklines tools on the market, as Excel's charting complexity is well known. Some Excel add-ins already helped in formatting charts automatically (ex: Peltier tech.), following Tufte's rules (ex: Juice Analytics), but these did not suit my needs.
Creating and resizing Excel charts using Visual Basic for Applications (VBA) had been intended, but with limited success, without simplifying the chart object itself.
My intention, based on Tufte's sparklines concept (word size graphics), combined with the grid structure of a spreadsheet, was to create tiny charts that would fit in a cell.
The main benefit of this approach, is that charts are ideally placed on the grid, aligned with other data in a table. One downside however, is that given their size, charts would be very simple and convey little information. This actually turned out to be a very positive point, as using "small multiples" makes the identification of patterns and correlations in datasets much easier.
I choose to create a VBA macro that would literally build a chart, piece by piece, by assembling tiny "shapes" (lines segments, rectangles, dots) inside the area of a spreadsheet cell. This approach implied that that had to figure out what are the components of a chart and how they are put together to form a meaningful picture.
After extensive research and despite of little experience in programming, I found out that the VBA "shape" object actually offers plenty of properties (Size, position and color to name a few) that

allow to accurately place shapes within a cell, using the bottom-left and top-right coordinates of the cell as reference points.

Outcome

I created a set of Excel formulas known as “User defined functions” (UDF), that, instead of returning text or figures in a cell would create a tiny chart.

Given the reduced size of the charts, every individual component of the graphic had to be carefully thought: Is this line absolutely necessary? What information does this line provide? Which parameters are required to control this element? Is the chart still readable? Which colors or thickness should be used?

For example, the formula “=linechart(A1,A20)” with values in cells A1 to A20, generates in an individual cell. Resizing the cell, automatically resizes the chart. Cells can be merged to make larger charts.

The formula LineChart() includes 17 different parameters (ex: color, thickness or style (Plain or dashed) for 2 series of data – see Appendix 2). To simplify the usability of the formula only 1 parameter is mandatory: the range of data the user needs to plot: =linechart(A1:A9).

The same logic applies today to 14 different types of micro-charts, accessible through a dedicated Excel menu, on which charts are organized by chart purpose: Performance, Evolution, Distribution, Correlation and Composition.

Benefits

<u>Reduced learning curve:</u> The key advantage of generating charts through formulas, is that SfE formulas behave like any standard Excel formula. Excel users immediately understand the principle of formulas with multiple parameters.

As such, SfE formulas can be copied and pasted, or dragged across a range. SfE functions can contain embedded formulas, or be embedded in other formulas.

With its dedicated menu to select charts, SfE blends into Excel’s User Interface, this means that the learning curve for intermediate Excel users is extremely short.

As stated earlier, usually only 1 to 3 parameters are mandatory for each function. There is no need for users to go through a heavy user manual before creating a chart (although SfE has its own visual user manual – see appendix 2).

<u>Carefully selected standard parameters</u>: SfE standard parameters are set in order to have a minimalist colorblind- friendly rendering by default. The charts look-&-feel is based on Tufte’s minimalist principles. Nevertheless, charts can be customized to meet users needs (corporate color palette for example).

<u>Scalable code:</u> In terms of programming, the VBA code is based on a library of objects w/ a common structure. This makes the code scalable and easy to audit. For instance, some SfE customers built their on type of chart that overlaps a Barchart() and a LineChart() in one single formula.

As the code is open-source, it is easy for (advanced) users to debug the code in Excel VBA Editor and share improvements with SfE community on the dedicated forum.

Downsides

<u>Security and distribution:</u> SfE is distributed as an .xlam add-in file. This implies that SfE has to be installed on each end-user laptop, which is often prevented by corporate security policies.

The development and maintenance of SfE’s code is complex as Excel is now available in 32 or 64 bits version, in a Mac version and online as Office365 version. Each version of Excel has its VBA peculiarities, therefore I limited SfE to PC 32 and 64 versions

<u>Rendering:</u> In a complex report, SfE is can create thousands of objects that Excel has to render on screen. Very complex reports can take 1 or 2 seconds to render. This is however less of an issue as common laptops have sufficient memory and CPU power to manage large datasets.

Competition: Although Excel is still the preferred tool to build dashboards for many users, tools like Tableau or PowerBI provide more robust and free alternatives to Excel, especially when dataset are available on cloud storage. Although these tools have their own learning curve, they are simple enough for most users.

Usage and Impact

I extensively used SfE for ad-hoc reports or to quickly generate charts for white papers. I systematically used SfE in quick analysis of raw datasets (Pareto, visual descriptive statistics), as for example monitoring parcel volumes per ship-methods or creating Project management reports for AMZL expansion in IT and ES

I still use SfE to wireframe reports before generating a PowerBi report or other online reporting tool.

The main impact though is that SfE was a project that satisfied my curiosity and allowed me to dive deep into data-visualization principles. I learned advanced VBA code by relentlessly facing hurdles and trying to find elegant coding solutions.

SfE is a free, open-source dataviz library. It has been downloaded more than 100.000 times and is still being used by Excel addicts in small businesses and large corporations.

Appendices

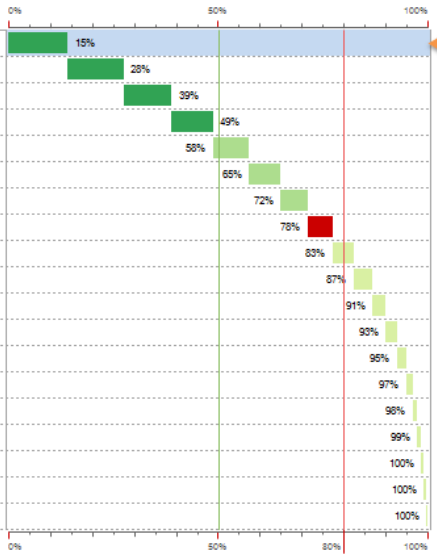
Appendix 1 portfolio of SFE charts generated by formulas

Appendix 2 SfE user manual

Appendix 1

Example of Pareto displayed with one single formula (blue cell)

Items	Quantity	Color Code	% of total
Item 1	80	5546801	14%
Item 2	75	5546801	13%
Item 3	65	5546801	12%
Item 4	56	5546801	10%
Item 5	49	9362861	9%
Item 6	42	9362861	7%
Item 7	36	9362861	6%
Item 8	34	9362861	6%
Item 9	28	10744025	5%
Item 10	26	10744025	5%
Item 11	18	10744025	3%
Item 12	15	10744025	3%
Item 13	12	10744025	2%
Item 14	9	10744025	2%
Item 15	6	10744025	1%
Item 16	5	10744025	1%
Item 17	4	10744025	1%
Item 18	3	10744025	1%
Item 19	2	10744025	0%



Pareto parameters

Target Line 1	50%
Target Line 2	80%
Highlight position #	8
☑ Legend	

With Pareto(), use ScaleXY().

Formula in cell F3

=pareto(B3:B21;C3:C21;C24;C25;C26)

Example of Dashboard using linechart(), Bulletchart() and Varichart()

Executive dashboard

this data is as of november 15th, 2005

Key Figures

bad ok good

	actuals nov '04 - oct '05	current month	% of prev. year	year to date	% of YTD target
Late arrivals		48	59%	714	179%
Numbers of minutes late		1,342	59%	15,363	188%
Cancelled flights		7	58%	104	80%
Flight utilization		94%	109%	91%	101%
Average customer satisfaction		4.5	100%	4.5	96%
Market share in %		47%	112%	44%	96%
Revenue in million		9.4	67%	175.2	96%
Expenses in million		6.6	71%	114.1	89%
Profits in million		2.8	59%	61.1	111%
Fuel costs in million		0.69	72%	12.8	116%
Revenue / passenger		289.6	110%	294.6	100%
Revenue / passenger mile		0.6	117%	0.6	113%
Profit / passenger mile		0.2	103%	0.2	131%
Profit / mile		17.0	115%	17.4	116%
Fuel costs / mile		4.2	142%	3.7	122%
Flights		231	54%	4,602	96%
Passengers in thousands		32	61%	595	95%
Passenger miles in million		15	57%	312	85%
Miles in thousands		163	51%	3,513	96%

0% 100% 200%

Revenues per Sales Channel

	actuals nov '04 - oct '05	current month	prev. year	year to date	% of revenue
Direct via Internet		4.5	72%	81.6	47%
Indirect via Internet		3.0	66%	57.2	33%
Direct via phone		1.2	58%	22.7	13%
Indirect via phone		0.4	61%	8.5	5%
Direct via counter		0.3	65%	5.2	3%

Cancel./delays by reason (last 30 days)

#	reason	cancelled	delayed
1	Weather	6	76
2	Mechanical failure	1	15
3	Missing or late flight crew	2	7
4	Missing or late ground crev	1	4
5	Inefficient gate handling	0	2
6	Other	2	3

Worse 10 routes (last 30 days)

#	from	to	cancelled in %	delayed in %
1	Detroit	Orlando	5.1%	31.4%
2	Chicago	Dallas	4.6%	26.3%
3	Minneapolis	Denver	4.2%	29.7%
4	Houston	Orlando	4.1%	21.7%
5	Chicago	Orlando	3.9%	25.6%
6	Memphis	Detroit	3.2%	15.8%
7	Salt Lake City	Boston	2.8%	19.7%
8	Oakland	Orlando	1.9%	14.9%
9	Dallas	Houston	1.1%	16.7%
10	Oakland	Seattle	0.9%	14.3%

Top 10 routes (last 30 days)

#	from	to	passengers in %	profit in %
1	Los Angeles	Oakland	12.6%	10.5%
2	Los Angeles	Dallas	9.7%	10.2%
3	Oakland	Dallas	8.2%	8.7%
4	Dallas	Houston	6.3%	7.5%
5	Oakland	Seattle	6.3%	6.9%
6	Houston	Orlando	3.9%	4.2%
7	Chicago	Dallas	2.6%	3.2%
8	Chicago	Orlando	2.0%	2.1%
9	Los Angeles	Orlando	2.1%	1.8%
10	Oakland	Orlando	1.9%	1.7%

Description of BoxPlotchart() function

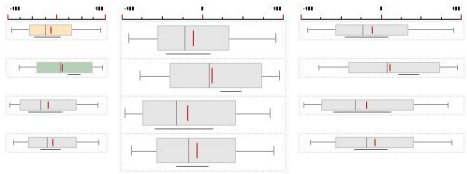
BoxPlotChart()

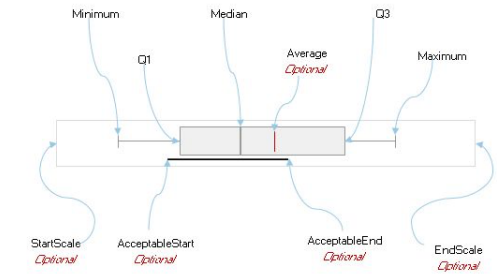
=boxplotchart(Minimum ; Q1 ; Median ; Q3 ; Maximum ; Average ; StartScale ; EndScale ; AcceptableStart ; AcceptableEnd ; ColorScheme)

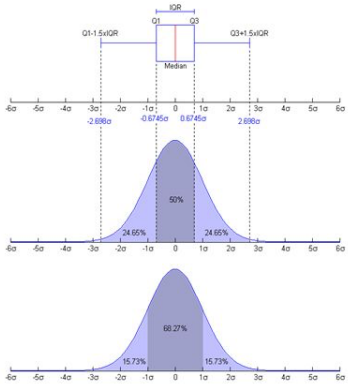
Minimum	Q1	Median	Q3	Maximum	Average	Acceptable Start	Acceptable End
-92	-56,5	-22,5	32	90	-11,3	-45,3	9,067
-78	-40,3	7	72,25	94	10,57	21,33	46,93
-97	-74,8	-33	40,5	83	-18,4	-59,3	11,87
-88	-57	-18	40,5	87	-7,82	-33	6,6

Start Scale -100 End Scale 100

Thanks to Nixnut for his contribution to the code.



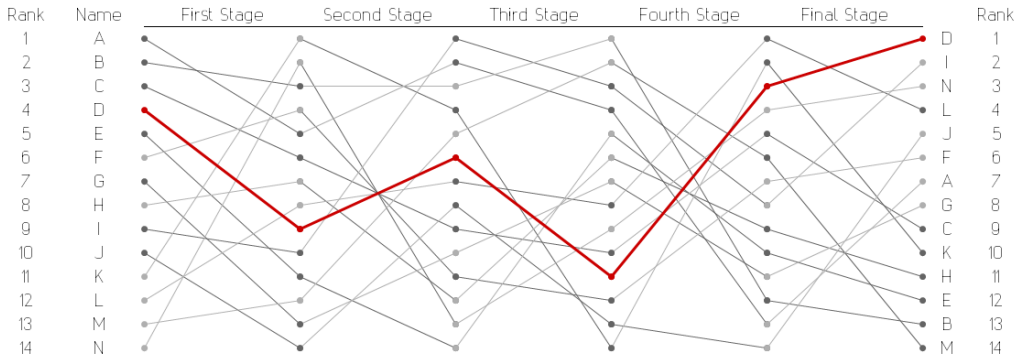




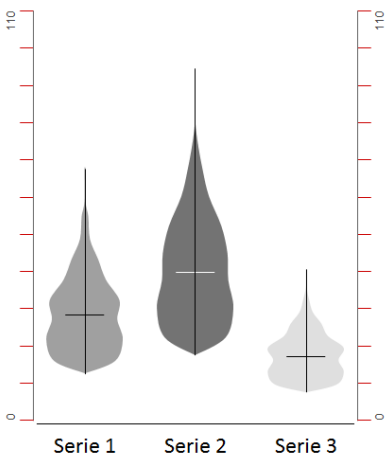
- First quartile (designated **Q1**) = lower quartile = cuts off lowest 25% of data = 25th percentile
- Second quartile (Q2) = **median** = cuts data set in half = 50th percentile
- Third quartile (designated **Q3**) = upper quartile = cuts off highest 25% of data, or lowest 75% = 75th percentile

- The difference between the upper and lower quartiles is called the interquartile range.

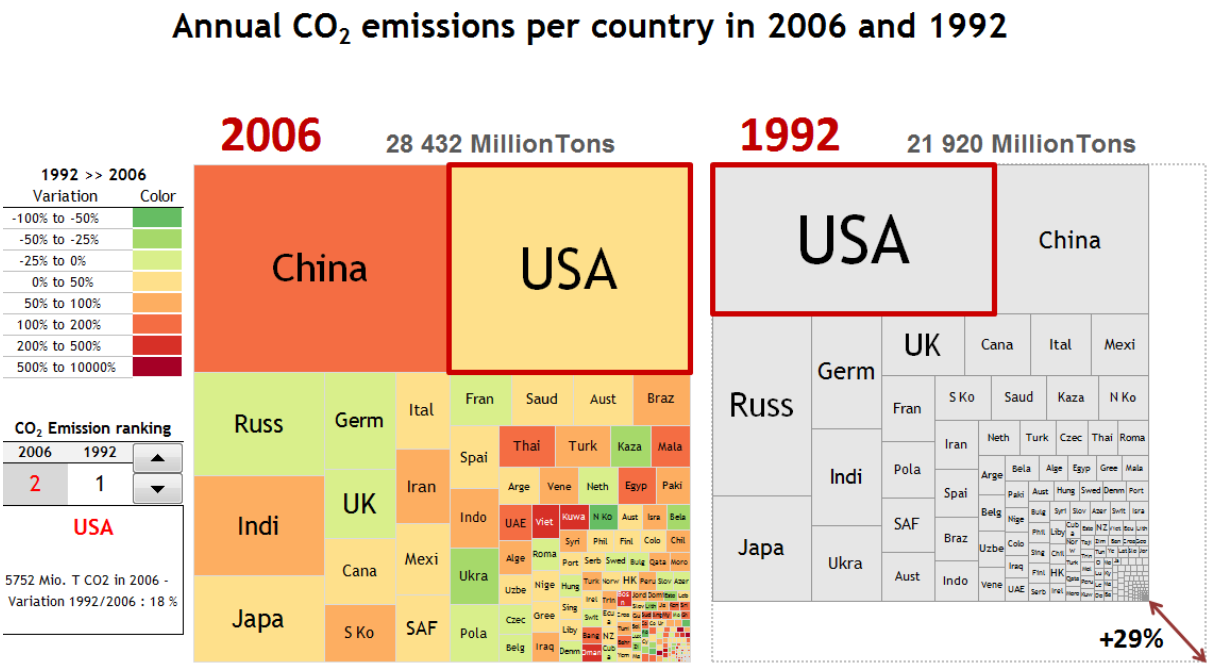
Output of the BumpChart() function. The Chart is built on 5 functions, one per column



Output of the BeanPLot() Function, showing the statistical distribution of a dataset with more precision than a BoxPlot()



Example of report based on the Treemap() function



Example of report using GanttChart() and CascadeChart() functions in parallel

Parameters

Project Start	01/01/2011
Project End	31/12/2011
Today	27/10/2011
Deadline 1	05/03/2011
Deadline 2	15/06/2011
Deadline 3	31/12/2011
Hit 1 Symbol	4
Hit 1 Color	203

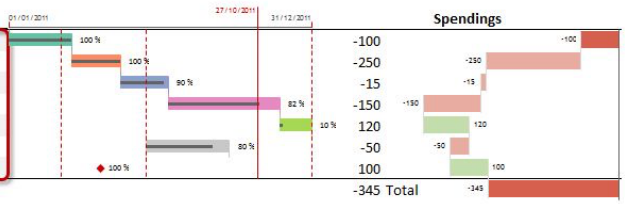
Colors
203
203
203
203

1,2,3 or 4

Gantt() formulas in H16 to H22
Pareto() formula in k16
ScaleXY() formula in H15

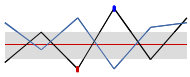
Hover over task to display tooltip
Click on task to follow hyperlink

Task Name	Task Start	Task End	Hit Date	Completion	Colors	Spendings
Task 1	01/01/2011	17/03/2011		100%	10 863 206	-100
Task 2	17/03/2011	15/05/2011		100%	6 458 876	-250
Task 3	15/05/2011	12/07/2011		90%	13 344 909	-15
Task 4	12/07/2011	23/11/2011		82%	12 815 079	-150
Task 5	23/11/2011	31/12/2011		10%	5 560 486	120
Task 6 + Hit 1	15/06/2011	23/09/2011	15/08/2011	80%	Default	-50
Hit 2	21/04/2011	21/04/2011	TaskStart=TaskEnd	100%		100
						-345 Total



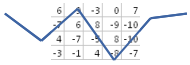
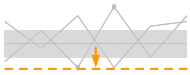
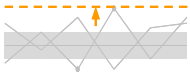
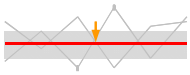
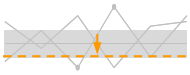
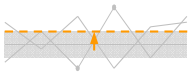
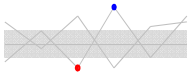
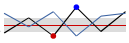
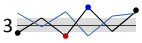
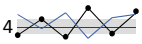
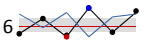
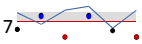
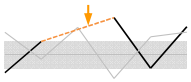
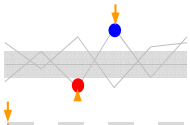
Appendix 2 – SfE Manual – Linechart() example

Sparklines for Excel v 3.7 - January 2010



LineChart()

=LineChart(Points1; Points2; Mini; Maxi; HLine; MinZone; MaxZone; Tags; Color1; Color2 ;...
GapStyle; TagMinColor; TagMaxColor; ShadedStart; ShadedWidth, TransparentWidth; MarkPosition)

	Points1	Range	Mandatory parameter
	Points2	Range	All other parameters are optional
	Mini	Variant	
	Maxi	Variant	
	HLine	Variant	Horizontal line
	MinZone	Variant	Bottom of the "normality" zone
	MaxZone	Variant	Top of the "normality" zone
	Tags	Integer	1  2 3  4  6  7 
	GapStyle	Integer	0 = don't interpolate and leave empty space 1 = interpolate and show solide line 2 = interpolate and show dotted line
	TagMinColor TagMaxColor	Long Long	Color code for the Minimum value tag (default = red) Color code for the Maximum value tag (default = blue)
	ShadedStart	Integer	Number of values before the first shaded segment in this example : 0 - Can take negative values
	ShadedWidth	Integer	Width of shaded segment in number of values in this example : 1 as grey area covers 1 point
	TranspWidth	Integer	Width of space between segments in number of values in this example : 1 as white area covers 1 point
	MarkPosition	Integer	Position of a vertical bar (item #) in this example : 2, as the bar is on top of the second point